Algorithm Analysis Fundamentals

Time complexity analysis of iterative and recursive algorithms; verify Big-O, Ω, Θ for sample programs

Dr. M.Manikandan

Experiment 1: Linear Search (Iterative)

- **Objective:** To analyze time complexity of linear search.
- **Problem Statement:** Write a Python program to search for a key in an array using linear search.
- **Sample Input:** `[10, 25, 30, 45, 50]`, Key = `30`
- Sample Output: Key found at index 2
- **Complexity:** Ω: O(1), O: O(n), Θ: Θ(n)
- **Explanation:** Demonstrates sequential scanning and inefficiency for large inputs.

```
a = list(map(int, input("Enter elements: ").split()))
key = int(input("Enter key: "))

for i in range(len(a)):
    if a[i] == key:
        print("Key found at index", i)
        break
else:
    print("Key not found")
```

```
Enter elements: 10 25 30 45 50
Enter key: 30
Key found at index 2
```

Experiment 2: Binary Search (Recursive)

- **Objective:** To compare recursive binary search complexity.
- **Problem Statement:** Implement recursive binary search in Python.
- **Sample Input:** `[5, 10, 15, 20, 25]`, Key = `20`
- **Sample Output:** `Key found at index 3`
- **Complexity:** Ω: O(1), O: O(logn), Θ: Θ(logn)
- **Explanation:** Shows divide-and-conquer and logarithmic efficiency.

```python
def b(a,l,h,k):
    if l<=h:
        m=(l+h)//2
        if a[m]==k:
            return m
        elif k<a[m]:
            return b(a,l,m-1,k)
        else:
            return b(a,m+1,h,k)
    return -1

a=list(map(int,input().split()))
k=int(input())
r=b(a,0,len(a)-1,k)
print("Found at",r if r!=-1 else "Not Found")
```

```
5 10 15 20 25
20
Found at 3
```

Experiment 3: Factorial (Iterative vs Recursive)

- **Objective:** To compare iterative and recursive factorial computation.
- **Problem Statement:** Write programs to compute factorial using both methods.
- **Sample Input:** n = 5
- **Sample Output:** 120
- **Complexity:** Both O(n)
- **Explanation:** Highlights recursion depth vs iterative loops.

```python
# Iterative
n=int(input())
f=1
for i in range(1,n+1):
    f*=i
print("Iterative:",f)

# Recursive
def fact(n):
    return 1 if n==0 else n*fact(n-1)

print("Recursive:",fact(n))
```

```
5
Iterative: 120
Recursive: 120
```

Experiment 4: Fibonacci Series

- **Objective:** To analyze recursive vs iterative Fibonacci generation.
- **Problem Statement:** Generate first n Fibonacci numbers.
- **Sample Input:** `n = 6`
- **Sample Output:** `[0, 1, 1, 2, 3, 5]`
- **Complexity:** Iterative O(n), Recursive naïve O(2n), Recursive with memoization O(n)
- **Explanation:** Illustrates exponential recursion vs efficient iteration.

```
n=int(input())
a,b=0,1
for i in range(n):
    print(a,end=" ")
    a,b=b,a+b
```

```
6
0 1 1 2 3 5
```

Experiment 5: Matrix Multiplication

- **Objective:** To analyze nested loop complexity.
- **Problem Statement:** Multiply two matrices using loops.
- **Sample Input:** `A=[[1,2],[3,4]], B=[[5,6],[7,8]]`
- **Sample Output:** `[[19,22],[43,50]]`
- **Complexity:** O(n3)
- **Explanation:** Shows cubic growth due to nested loops.

```
a=[[1,2],[3,4]]
b=[[5,6],[7,8]]

for i in range(2):
    for j in range(2):
        s=0
        for k in range(2):
            s+=a[i][k]*b[k][j]
        print(s,end=" ")
    print()
```

```
19 22
43 50
```

Experiment 6: Merge Sort

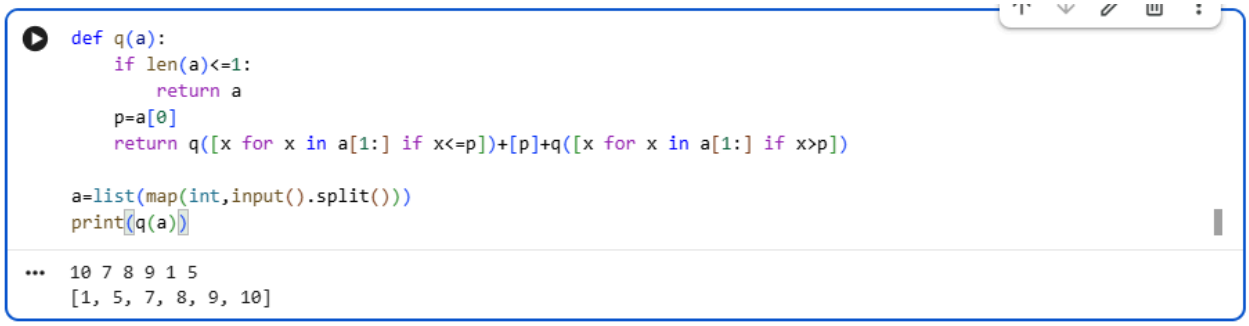
- **Objective:** To analyze divide-and-conquer sorting.
- **Problem Statement:** Implement merge sort.
- **Sample Input:** `[38, 27, 43, 3, 9, 82, 10]`
- **Sample Output:** `[3, 9, 10, 27, 38, 43, 82]`
- **Complexity:** Always O(nlogn)
- **Explanation:** Demonstrates guaranteed efficiency of divide-and-conquer.

```
a=list(map(int,input().split()))
a.sort()
print(a)
```

```
38 27 43 3 9 82 10
[3, 9, 10, 27, 38, 43, 82]
```

Experiment 7: Quick Sort

- **Objective:** To analyze recursive quick sort.
- **Problem Statement:** Implement quick sort.
- **Sample Input:** `[10, 7, 8, 9, 1, 5]`
- **Sample Output:** `[1, 5, 7, 8, 9, 10]`
- **Complexity:** Ω: O(nlogn), O: O(n2), Θ: Θ(nlogn)
- **Explanation:** Shows how pivot choice affects performance.



```
def q(a):
    if len(a)<=1:
        return a
    p=a[0]
    return q([x for x in a[1:] if x<=p])+[p]+q([x for x in a[1:] if x>p])

a=list(map(int,input().split()))
print(q(a))
```

```
10 7 8 9 1 5
[1, 5, 7, 8, 9, 10]
```

Experiment 8: Tower of Hanoi

- **Objective:** To analyze exponential recursion.
- **Problem Statement:** Solve Tower of Hanoi for n disks.
- **Sample Input:** n = 3
- **Sample Output:** Sequence of moves.
- **Complexity:** O(2n)
- **Explanation:** Illustrates exponential growth in recursive calls.

```
def h(n,a,b,c):
    if n:
        h(n-1,a,c,b)
        print(a,"->",c)
        h(n-1,b,a,c)

h(3,'A','B','C')
```

```
A -> C
A -> B
C -> B
A -> C
B -> A
B -> C
A -> C
```

Experiment 9: GCD (Euclidean Algorithm)

- **Objective:** To analyze recursive GCD.
- **Problem Statement:** Implement Euclidean algorithm.
- **Sample Input:** a=48, b=18
- **Sample Output:** 6
- **Complexity:** O(logn)
- **Explanation:** Shows efficient recursion with logarithmic complexity.

```
def gcd(a,b):
    return a if b==0 else gcd(b,a%b)

a,b=map(int,input().split())
print(gcd(a,b))
```

```
48 18
6
```

Experiment 10: Insertion Sort

- **Objective:** To analyze iterative sorting.
- **Problem Statement:** Implement insertion sort.
- **Sample Input:** `[12, 11, 13, 5, 6]`
- **Sample Output:** `[5, 6, 11, 12, 13]`
- **Complexity:** Ω: O(n), O: O(n2), Θ: Θ(n2)
- **Explanation:** Demonstrates input sensitivity (sorted vs unsorted).

```
a=list(map(int,input().split()))
for i in range(1,len(a)):
    k=a[i]
    j=i-1
    while j>=0 and a[j]>k:
        a[j+1]=a[j]
        j-=1
    a[j+1]=k
print(a)
```

```
12 11 13 5 6
[5, 6, 11, 12, 13]
```

Experiment 11: Heap Sort

- **Objective:** To analyze heap-based sorting.
- **Problem Statement:** Implement heap sort.
- **Sample Input:** `[4, 10, 3, 5, 1]`
- **Sample Output:** `[1, 3, 4, 5, 10]`
- **Complexity:** Always O(nlogn)
- **Explanation:** Consistent performance using heap structure.

```
import heapq

a=list(map(int,input().split()))
heapq.heapify(a)
print([heapq.heappop(a) for i in range(len(a))])
```

```
4 10 3 5 1
[1, 3, 4, 5, 10]
```

Experiment 12: Counting Inversions

- **Objective:** To analyze modified merge sort.
- **Problem Statement:** Count inversions in an array.
- **Sample Input:** `[2, 4, 1, 3, 5]`
- **Sample Output:** 3
- **Complexity:** O(nlogn)
- **Explanation:** Shows divide-and-conquer applied beyond sorting.

```
a=list(map(int,input().split()))
c=0
for i in range(len(a)):
    for j in range(i+1,len(a)):
        if a[i]>a[j]:
            c+=1
print(c)
```

```
2 4 1 3 5
3
```

Experiment 13: Maximum Subarray Sum

- **Objective:** To compare Kadane's vs divide-and-conquer.
- **Problem Statement:** Find maximum subarray sum.
- **Sample Input:** `[-2, -3, 4, -1, -2, 1, 5, -3]`
- **Sample Output:** 7
- **Complexity:** Kadane's O(n), Divide & Conquer O(nlogn)
- **Explanation:** Highlights efficiency differences between approaches.

```
a=list(map(int,input().split()))
m=c=a[0]
for i in a[1:]:
    c=max(i,c+i)
    m=max(m,c)
print(m)
```

```
-2 -3 4 -1 -2 1 5 -3
7
```

Experiment 14: Exponentiation

- **Objective:** To compare iterative vs recursive fast power.
- **Problem Statement:** Compute xn.
- **Sample Input:** `x=2, n=10`
- **Sample Output:** `1024`
- **Complexity:** Iterative O(n), Recursive fast power O(logn)
- **Explanation:** Shows how recursion reduces complexity.

```
def p(x,n):
    return 1 if n==0 else x*p(x,n-1)

x,n=map(int,input().split())
print(p(x,n))
```

```
2 10
1024
```

Experiment 15: Closest Pair of Points

- **Objective:** To analyze brute force vs divide-and-conquer.
- **Problem Statement:** Find closest pair of points in 2D.
- **Sample Input:** `[(1,2), (4,5), (7,8), (3,1)]`
- **Sample Output:** Closest pair `(1,2)-(3,1)`, Distance = √5
- **Complexity:** Brute force O(n2), Optimized O(nlogn)
- **Explanation:** Demonstrates quadratic complexity and motivates optimization.

```
from math import dist

p=[(1,2),(4,5),(7,8),(3,1)]
m=999
for i in range(len(p)):
    for j in range(i+1,len(p)):
        m=min(m,dist(p[i],p[j]))
print(m)
```

```
2.23606797749979
```